UNIVERSITATEA
„ALEXANDRU IOAN CUZA“
din IAŞI

Calcul Cuantic - Quantum Computing

Cursul 13 - Estimarea fazei. Quantum Factorization

Conf. dr. Andreea Arusoaie

e-mail: andreea.arusoaie at uaic.ro / info.uaic.ro

Web: http://edu.info.uaic.ro/quantum-computing

9 Ianuarie, 2025

Structura cursului

1. Transformata Fourier
   - DFT/IDFT
   - QFT
2. Quantum Phase estimation - Estimarea fazei
3. Factorizarea numerelor întregi
   - Modular Arithmetic
   - Găsirea ordinului unui număr întreg
   - Problema factorizării - varianta cuantică
   - Algoritmul de factorizare al lui Shor
   - Găsirea ordinului/perioadei

Introducere

Transformata Fourier - este o operație care se aplică unei funcții complexe și produce o altă funcție complexă care conține aceeași informație ca funcția originală, dar reorganizată după frecvențele componente.

Transformata Fourier Discretă (DFT) transformă un vector de N numere complexe $(x_0, \ldots, x_{N-1})$ într-un nou vector $(y_0, \ldots, y_{N-1})$, după formula:

$$y_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} \exp\left(2\pi i \cdot \frac{j \cdot k}{N}\right) x_j,$$

unde

- x_j este semnalul original în domeniul timpului.
- y_k sunt coeficienţii frecvenței obținuţi prin DFT.
- $\omega = \exp\left(\frac{2\pi i}{N}\right)$.
- N este numărul total de puncte din semnalul discret.

Transformata Fourier Discretă

Dacă notăm $\omega = \exp\left(\frac{2\pi i}{N}\right)$ atunci vom putea scrie suma pentru DFT astfel

$$y_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} \omega^{jk} x_j.$$

IDFT (Inverse Discrete Fourier Transform): este procesul prin care recuperăm semnalul discret x_j din coeficienţii y_k obținuti prin DFT.

$$x_j = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \omega^{-jk} y_k.$$

Unde:

- x_j este semnalul original în domeniul timpului.
- y_k sunt coeficienţii frecvenței obținuţi prin DFT.
- $\omega = \exp\left(\frac{2\pi i}{N}\right)$.
- N este numărul total de puncte din semnalul discret.

Transformata Fourier Cuantică

Transformata Fourier cuantică (QFT) este implementarea cuantică a DFT. Quantum Fourier Transform constituie parte importantă din mulţi algoritmi cuantici cunoscuţi şi importanti.

Transformata Fourier cuantică (QFT) aplicată stării $|\psi\rangle$ va fi egală cu

$$QFT|\psi\rangle = \sum_{k=0}^{N-1} y_k |k\rangle$$

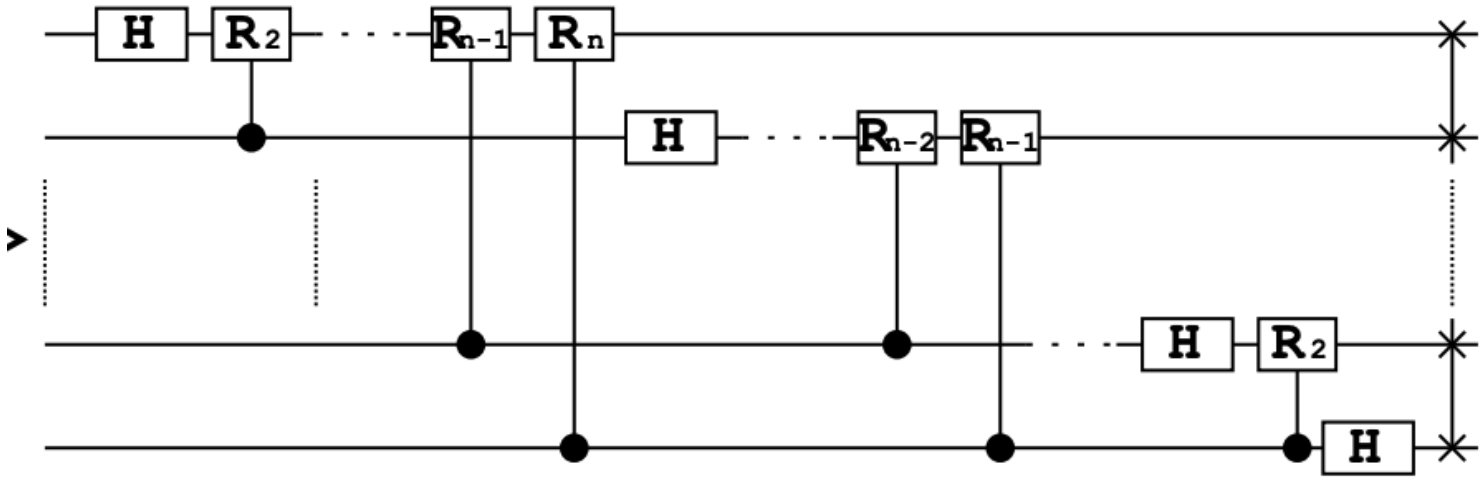
unde

$$y_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} \alpha_j \omega_N^{jk}$$

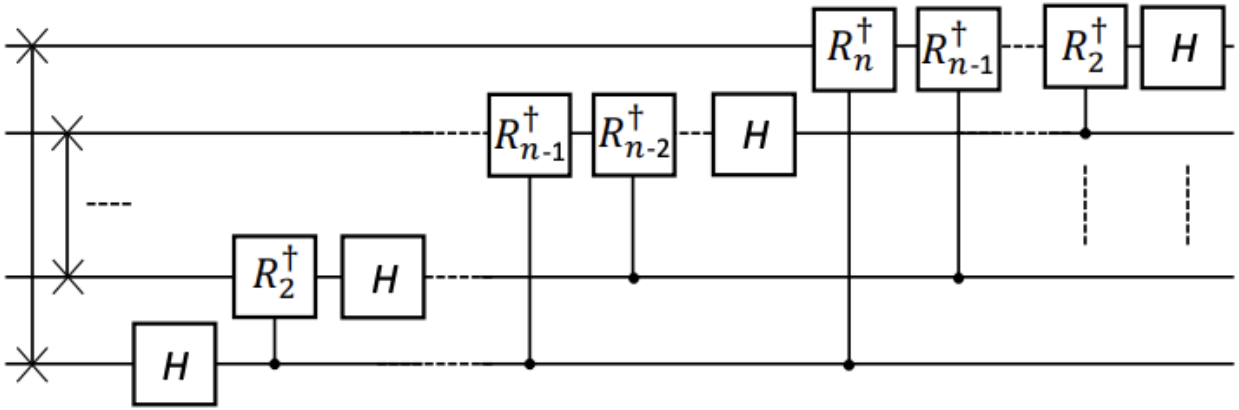
unde $\omega_N^{jk} = \exp\left(2\pi i \cdot \frac{jk}{N}\right)$ iar $|k\rangle, k = \{0, N-1\}$ reprezintă stările din baza computaţională de n qubiţi.

Transformata Fourier Cuantică

Transformata Fourier cuantică poate fi implementată astfel:



Transformata Fourier Inversă este implementată de urmatorul circuit:



Structura cursului

1. Transformata Fourier
   - DFT/IDFT
   - QFT
2. Quantum Phase estimation - Estimarea fazei
3. Factorizarea numerelor întregi
   - Modular Arithmetic
   - Găsirea ordinului unui număr întreg
   - Problema factorizării - varianta cuantică
   - Algoritmul de factorizare al lui Shor
   - Găsirea ordinului/perioadei

Quantum Phase estimation

Quantum Phase Estimation - QPE este un algoritm fundamental în calculul cuantic, utilizat pentru a determina faza (sau valoarea proprie unitară) asociată cu un operator unitar U.

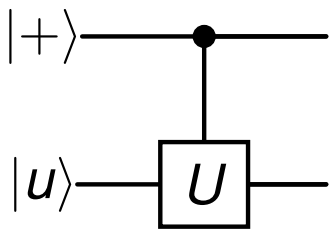
Acesta este esențială pentru multe aplicații importante în calculul cuantic, cum ar fi simularea sistemelor cuantice, rezolvarea de probleme legate de valori proprii și factorizarea numerelor în algoritmul lui Shor.

Problema:

Să presupunem că avem un oracol U ce acţionează pe n-qubiti, implementat de o poartă Controlled $-$ U. Ştim că U are valoarea proprie $e^{2\pi i\phi}$, şi, respectiv vectorul propriu asociat $|u\rangle$.
Vrem să determinăm faza ϕ a valorii proprii, cu o precizie de t biti.

QPE- actiunea oracolului

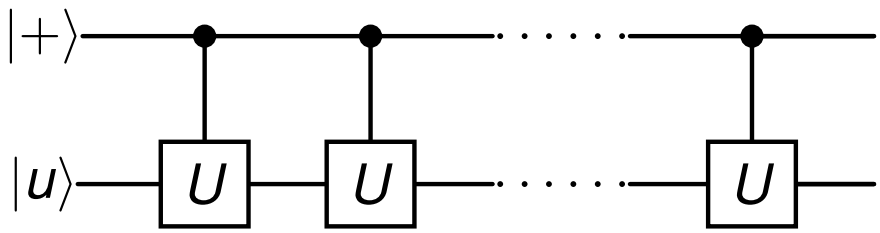
Să considerăm acţiunea porţii *Controlled* $-$ *U* pentru cazul când qubitul de control este in starea $|+\rangle$:



Astfel vom avea transformarea

$$|+\rangle \otimes |u\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |u\rangle \rightarrow \quad \frac{1}{\sqrt{2}}(|0\rangle\,|u\rangle + |1\rangle\, e^{2\pi i\phi}\,|u\rangle)$$
$$= \frac{1}{\sqrt{2}}(|0\rangle + e^{2\pi i\phi}\,|1\rangle)\,|u\rangle$$

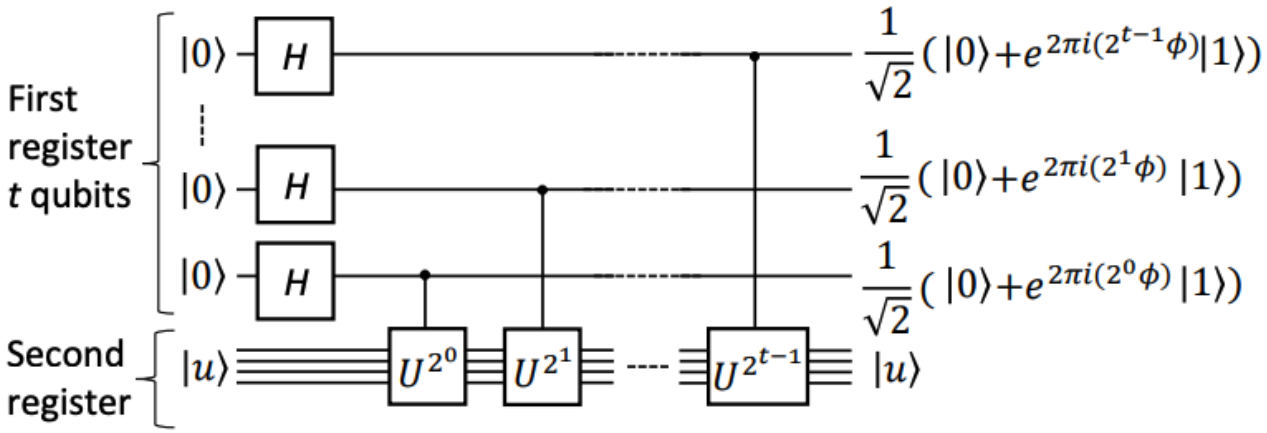
Aplicând *U* succesiv de *m* ori vom avea



$$|+\rangle \otimes |u\rangle \xrightarrow{C-U^m} \frac{1}{\sqrt{2}}(|0\rangle + e^{2m\pi i\phi}\,|1\rangle) \otimes |u\rangle\,.$$

Folosirea oracolului pentru a pregăti starea

Am vazut mai sus cum acţionează oracolul implementat de $Controlled - U$ asupra stării $|+\rangle \otimes |u\rangle$. În continuare, pentru a estima faza, îl vom aplica pentru un registru de t qubiti



Astfel, în urma circuitului vom avea

$$|0\rangle^t \otimes |u\rangle \longrightarrow \frac{1}{\sqrt{2^t}} \sum_{j=0}^{2^t-1} e^{2\pi i \phi j} |j\rangle \otimes |u\rangle$$

Folosirea IQFT pentru estimarea fazei

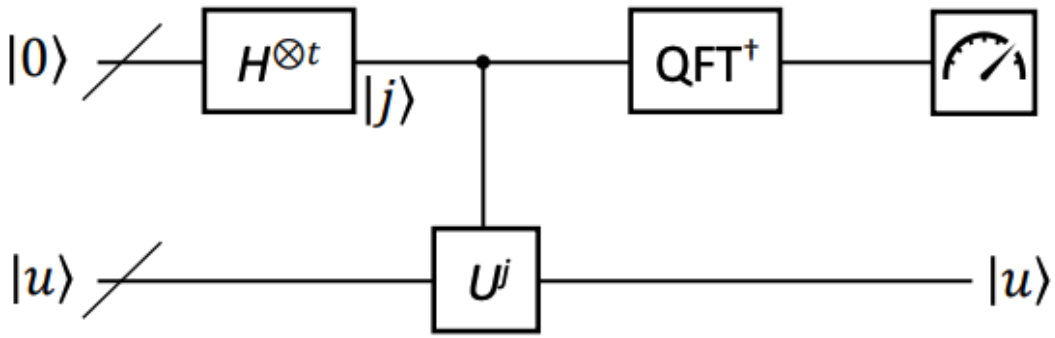
În continuare, vom aplica Transformata Fourier cuantică inversa (IQFT) asupra primului registru pentru a estima faza valorii proprii.

Aşadar, aplicand IQFT vom avea

$$\frac{1}{\sqrt{2^t}} \sum_{j=0}^{2^t-1} e^{2\pi i \phi j} \left|j\right\rangle \otimes \left|u\right\rangle \xrightarrow{QFT^\dagger} \left|\tilde{\phi}\right\rangle \otimes \left|u\right\rangle .$$

Deci, măsurând primul registru, vom obţine o aproximare de t-biti, dată de $\tilde{\phi}$, a valorii proprii ϕ.

Sumarizând, circuitul care face QPE este dat de



QPE - un exemplu

În practică, pentru eficienta, QPE se aplică unor operatori unitari de dimensiuni mari, însă pentru simplitate vom lua un exemplu ce actionează pe un qubit.

Considerăm $U = Y$, deci fie poarta

$$U = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}.$$

Se poate arăta că valorile proprii ale lui U sunt ± 1.

Corespunzător lui $\lambda = -1$, avem vectorul propriu

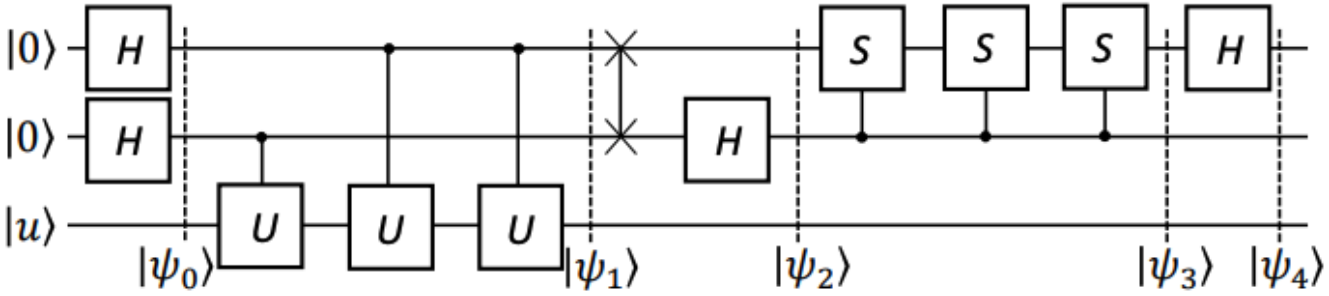
$$|u\rangle = \begin{pmatrix} 1 \\ -i \end{pmatrix} \text{ sau } |u\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -i \end{pmatrix} = \frac{1}{\sqrt{2}}(|0\rangle - i\,|1\rangle).$$

care este corespunzător fazei $\phi = \frac{1}{2}$.

Vrem să găsim faza cu doi biti de precizie $t = 2$, folosind QPE.

QPE - un exemplu

Pentru $t = 2$ vom considera următorul circuit



$$|\psi_0\rangle = \tfrac{1}{2}(|0\rangle + |1\rangle)(|0\rangle + |1\rangle)\,|u\rangle$$

$$|\psi_1\rangle = \tfrac{1}{2}(|0\rangle + e^{2\pi i}\,|1\rangle)(|0\rangle + e^{\pi i}\,|1\rangle)\,|u\rangle = \tfrac{1}{2}(|0\rangle + |1\rangle)(|0\rangle - |1\rangle)\,|u\rangle$$

$$|\psi_2\rangle = \tfrac{1}{\sqrt{2}}(|0\rangle - |1\rangle)\,|0\rangle\,|u\rangle$$

$$|\psi_3\rangle = \tfrac{1}{\sqrt{2}}(|0\rangle - |1\rangle)\,|0\rangle\,|u\rangle$$

$$|\psi_4\rangle = |1\rangle\,|0\rangle\,|u\rangle\,.$$

Deci măsurăm primii doi qubiti şi obţinem $|10\rangle$, ce corespunde fractiei binare:

$$1 \times \frac{1}{2} + 0 \times \frac{1}{2^2} = \frac{1}{2}.$$

QPE - Concluzii

- QPE este o componentă fundamentală a multor algoritmi cuantici.
- Permite determinarea fazei unei valori proprii asociate unui operator liniar cu o precizie arbitrară.
- QPE functionează eficient chiar şi atunci când faza nu poate fi exprimată ca o fractie binară.
- QPE are un rol esential în algoritmul lui Shor, pentru factorizarea numerelor întregi. În alg. lui Shor, QPE este folosit pentru a găsi perioada unei funcţii.

Structura cursului

1. Transformata Fourier
   - DFT/IDFT
   - QFT
2. Quantum Phase estimation - Estimarea fazei
3. Factorizarea numerelor întregi
   - Modular Arithmetic
   - Găsirea ordinului unui număr întreg
   - Problema factorizării - varianta cuantică
   - Algoritmul de factorizare al lui Shor
   - Găsirea ordinului/perioadei

Factorizarea numerelor întregi

Problema factorizării constă în găsirea factorilor primi ai unui număr întreg, pozitiv, N.

Teorema fundamentală a Aritmeticii spune că orice număr întreg, pozitiv, N admite o descompunere în factori primi, i.e.

$$N = p_1^{k_1} \cdot p_2^{k_2} \cdot \ldots \cdot p_m^{k_m}.$$

unde p_i sunt numere prime distincte iar k_i sunt numere întregi pozitive. Mai mult, această descompunere este *unică*.

Exemplu: Să se găsească factorii primi ai numărului 180.

Factorizarea numerelor întregi

Cel mai dificil caz în problema factorizării este atunci când

$$N = p \cdot q,$$

unde p şi q sunt numere prime mari.

Spre exemplu dacă $N = 41449$, cum îi găsesc factorii primi p şi q astfel încât $p \cdot q = 41449$?

Date numerele întregi prime p şi q, este uşor de calculat $N = p \cdot q$, însă dat numărul întreg N, este dificil de găsit factorii primi p, q.

Dacă ştiu $p = 181$, pot să îl găsesc uşor şi pe $q = 229$.

Factorizarea numerelor întregi

Cel mai eficient algoritm clasic care rezolvă problema factorizării pentru cazul numerelor mari este *number field sieve (GNFS)*.

Numărul de operaţii necesare pentru a rezolva problema factorizării utilizând acest algoritm este $\exp\left(\mathcal{O}\left(n^{\frac{1}{3}}(\ln n)^{\frac{2}{3}}\right)\right)$, unde $n = \lceil \log_2(N) \rceil$ este numărul de biţi necesari pentru a reprezentarea numărului N.

- Factorizarea unui număr de 400 digits ar dura 10^{10} ani.

Problema factorizării este importantă în practică pentru că pe dificultatea factorizării se bazează majoritatea schemelor de criptare.

Modular Arithmetic

Fie $N > 1$ un număr întreg.

- Spunem că două numere întregi a şi b sunt congruente modulo N, dacă N divide $a - b$, adică există k astfel încât $a - b = kN$.
- Vom nota congruenţa modulo N astfel: $a \equiv b \ (mod\ N)$

Exemple:

- Fie $\mathbb{Z}_5 = \{\widehat{0}, \widehat{1}, \widehat{2}, \widehat{3}, \widehat{4}\}$. Atunci $7 \equiv 2(mod\ 5)$.
- Dacă $N = 21$ atunci $24 \equiv 3(mod\ 21)$, $20 \equiv -1(mod\ 21)$.
- $24 \cdot 26(mod 21) = (3 mod 21) \cdot (5 mod 21) = 15 mod 21$.
- Ordinul lui 2 în $\mathbb{Z}_5$ este $r \in \mathbb{N}$ astfel încât $2^r \equiv 1(mod 5) \Rightarrow r = 4$.

Criptarea RSA - pe scurt

Criptosistemele *RSA* (Rivest-Shamir-Adleman - 1977) sunt printre cele mai des utilizate metode de criptare cu cheie publică.

Acest protocol utilizează două chei: una publică şi una privată, şi se bazează pe dificultatea factorizării numerelor mari prime.

Algoritmul RSA generează chei de 2048 biti după următoarea schemă:

1. Generează p şi q numere prime mari;
2. Calculează $N = p \cdot q$;
3. Calculează funcţia totient a lui Euler: $\phi(N) = (p-1)(q-1)$;
4. Se alege un număr e, coprim cu $\phi(N)$ ($gcd(\phi(N), e) = 1$) şi se publică perechea (N, e) - cheia publică;
5. Se determină cheia privată, adică se găste $d \in \mathbb{Z}$ astfel încât

$$d \cdot e \equiv 1 (mod\ \phi),$$

(N, d) - va fi cheia privată.
Pentru a cripta/decripta mesaje, se vor utiliza N, d, e.

Criptarea RSA - pe scurt

Unele site-uri, doresc ca tu sa poţi să le trimiţi un mesaj pe care doar ei să îl poată decripta (datele de pe cardul de credit).

Prin urmare, acestia vor găsi două numere mari prime p şi q pe care le vor ţine secrete, le va înmulţi pentru a obţine $N = pq$, şi va publica N.

Iar noi vom putea cripta un mesaj folosind o cheie publică N astfel: dacă mesajul este x, atunci folosind cea mai simplă versiune de *RSA* mesajul criptat va fi x^3 mod N.

Cei de la site, ştiind numerele prime p şi q şi mesajul criptat, va putea recupera mesajul x folosind câteva noţiuni de bază din teoria numerelor.

Aşadar, ideea de bază a securităţii de acest tip este aceea că dacă ştim p şi q, atunci vom şti ordinea grupului multiplicativ modulo N, şi, prin urmare, îl voi putea găsi pe x.

Însă dacă nu ştim p, q ci ştim doar produsul N, problema va deveni mult mai complicată.

Algoritmul lui Euclid

- Algoritmul lui Euclid se utilizează pentru găsirea celui mai mare divizor comun al numerelor întregi p şi q, notat $gcd(p, q)$.

Teoremă: Dacă p şi q sunt numere întregi, iar $r \in \mathbb{Z}$, $r \neq 0$ este restul împărţirii lui p la q, atunci $gcd(p, q) = gcd(q, r)$.

Algoritmul lui Euclid de găsire a $gcd(p, q)$ cu $p > q$ are următorii paşi:

- se împarte p la q pentru a găsi restul $r_1 < q$ şi vom avea relaţia $gcd(p, q) = gcd(q, r_1)$;
- se împarte q la r_1 şi se obţine $r_2 < r_1$, în plus, are loc $gcd(q, r_1) = gcd(r_2, r_1)$;
- repetând procedeul vom divide r_{n-2} la r_{n-1} şi vom găsi restul $r_n < r_{n-1}$. Mai mult vom avea, $gcd(p, q) = \ldots = gcd(r_{n-1}, r_n)$
- cum r_i este o secvenţă strict descrescătoare de numere pozitive, va exista n cu $r_{n+1} = 0$, adică $r_{n-1} = \alpha r_n$.
- algoritmul se termină dacă avem $gcd(p, q) = gcd(r_{n-1}, r_n) = r_n$.

Algoritmul lui Euclid

Exemplu: Determinaţi $gcd(6825, 1430)$.

6825=4· 1430+1105	gcd(6825,1430)=gcd(1430,1105)
1430=1· 1105+325	gcd(1430,1105)=gcd(1105,325)
1105=3· 325+130	gcd(1105,325)=gcd(325,130)
325=2· 130+65	gcd(325,130)=gcd(130,65)
130=2· 65+0	gcd(6825,1430)=gcd(130,65)=65

Observaţie:

- Dacă p şi q sunt numere întregi reprezentaţe cu n biţi, atunci r_i va fi un întreg tot din n biţi, pentru orice indice i.
- Fiecare împărţire va va costa $\mathcal{O}(n^2)$, iar cum $r_{i+2} \leq \frac{r_i}{2}$, vom avea nevoie de $\mathcal{O}(n)$ împărţiri; deci $gcd(p, q)$ va costa $\mathcal{O}(n^3)$.

Găsirea ordinului unui număr întreg

Fie x şi N numere întregi pozitive. Atunci, vom distinge următoarele situaţii

- dacă x şi N au factori comuni, atunci $gcd(x, N)$ este factor al lui N; astfel vom considera doar cazul când x este coprim cu N(adică $gcd(x, N) = 1$);
- dacă N este par, îl vom împărţi la 2 repetat până obţin un număr impar, apoi aplic algoritmul.

Definiţie

Numim ordinul lui x modulo N este cel mai mic număr întreg pozitiv r pentru care

$$x^r \equiv 1 \bmod N$$

- N este divizorul/factorul lui $x^r - 1$.

- Există mereu ordinul unui număr întreg deoarece puterile întregi x^k formează un grup ciclic finit.

Găsirea ordinului unui număr întreg

Dacă r este par, definim y prin $x^{r/2} \equiv y \bmod N$, adică restul împărţirii lui $x^{r/2}$ la N ($0 \leq y < N$).

Cum $y^2 \equiv 1 \bmod N$, vom obţine

$$x^r - 1 = (x^{r/2} - 1)(x^{r/2} + 1) = (y - 1)(y + 1) \equiv 0 \bmod N$$

deci N divide $(y - 1)(y + 1)$.

Cum $1 < y < N - 1$, vom avea că $0 < y - 1 < y + 1 < N$, deci N nu poate divide $y - 1$ şi $y + 1$ separat, ci N va avea un factor comun netrivial cu fiecare dintre $y - 1$ şi $y + 1$, factori care să dea N prin înmulţire.

Aşadar $gcd(y - 1, N)$ şi $gcd(y + 1, N)$ sunt factori netriviali ai lui N, şi pot fi calculaţi într-un număr de $\mathcal{O}(\log^3 N)$ operaţii.

Factorizarea numerelor întregi - varianta clasică

Sumarizând, problema factorizării întregilor are următorii paşi:

- Alegem un număr aleator x, cu $x < N$;
- Calculez $gcd(x, N)$
- Dacă $gcd(x, N) \neq 1$ am găsit un factor netrivial al lui N, şi mă opresc;
- Dacă $gcd(x, N) = 1$, găsesc ordinul r al lui x;
- Dacă r este par şi $0 < y - 1 < y + 1 < N$, factorii lui N sunt $gcd(y - 1, N)$ şi $gcd(y + 1, N)$.
- Dacă nu are loc una din condiţiile de la pasul precedent, mă întorc la pasul 1 şi aleg un alt x.

Observaţii:

1. Se poate întâmpla ca algoritmul să dea factori triviali ai lui N(1 şi N).
2. Algoritmul nu este eficient pentru N mare. Este dificil să calculăm perioada r al lui x mod N pentru N mare.
3. Cel mai eficient algoritm clasic care rezolvă problema factorizării este *general number field sieve (GNFS)* - $\mathcal{O}\left(\exp^{1.9\left(n^{\frac{1}{3}}(\ln n)^{\frac{2}{3}}\right)}\right)$, unde $n = \lceil \log_2(N) \rceil$.

Algoritmul de factorizare al lui Shor

Peter Shor

- În 1994, Peter Shor a propus un algoritm cuantic care poate factoriza numerele întregi în timp polinomial.
- Acest algoritm este probabil una dintre cele mai importante descoperiri din domeniul calculului cuantic.

Algoritmul de factorizare al lui Shor

Algoritmul propus de către Shor poate factoriza numărul N utilizând $\mathcal{O}(n^2 \log(n) \log(\log(n))$ operaţii, unde $n = \lceil \log_2(N) \rceil$;

- un număr de 400 digits va putea fi factorizat în mai puţin de 3 ani.

Algoritmul lui Shor este mai complex comparativ cu algoritmii studiati pana acum pentru că are şi părţi clasice si părţi cuantice.

Ideea ce stă la baza algoritmului este găsirea perioadei unei funcţii.

Algoritmul lui Shor a fost implementat la IBM în 2001, pentru factorizarea lui 15, utilizând un computer de 7 qubiti.

Problema factorizării - varianta cuantică

Algoritmul lui Shor conţine atat părţi clasice cât şi parti cuantice:

1. Alegem un număr aleator x, cu $x < N$;
2. Calculez $gcd(x, N)$; (Algoritmul lui Euclid)
3. Dacă $gcd(x, N) \neq 1$ am găsit un factor netrivial al lui N, şi mă opresc;
4. Dacă $gcd(x, N) = 1$, găsesc ordinul r al lui x, adică perioada următoarei funcţii $f(a) = x^a (mod\ N)$, i. e. cel mai mic număr pentru care $f(x + r) = f(x)$. (vom găsi ordinul utilizând un algoritm cuantic)
5. Dacă r este par: calculez $y = x^{r/2} \bmod N$:
   - dacă $y + 1 \equiv 0 \mod N$, ne întoarcem la pasul 1;
   - în caz contrar, calculez $gcd(y + 1, N)$ şi $gcd(y - 1, N)$;
   - dacă obţinem factori netriviali ai lui N, am rezolvat problema.

Problema factorizării - varianta cuantică

- Algoritmul cuantic propus de Peter Shor, calculează mult mai eficient ordinul/perioada unui număr întreg x şi coprim cu N folosind Transformata Fourier Cuantică (QFT).
- Algoritmul de găsire a perioadei al lui Shor se bazează pe abilitatea unui calculator cuantic de a fi în mai multe stări, în acelaşi timp (superpoziţie de stări).
- Pentru a calcula perioada unei funcţii, vom evalua funcţia în mai multe puncte, simultan.
- În mod clasic este greu de găsit perioada deoarece avem nevoie de multe evaluări ale funcţiei $f(t)$. Avantajul calculatorului cuantic este că el poate evalua funcţia în mai multe puncte, simultan.

Găsirea ordinului/perioadei

Dacă vom alege $f(t) = x^t \bmod N$, unde N este numărul de factorizat, iar $x < N$ este un număr ales aleator, atunci perioada r se calculează astfel:

- Pentru $N = 15$ şi $x = 2$ vom avea $f_{2,15}(t) = 2^t \bmod 15$ şi deci următoarele valori

t	0	1	2	3	4	5	6	7	8	9	10	11	12	...
$f_{2,15}(t)$	1	2	4	8	1	2	4	8	1	2	4	8	1	...

deci perioada r pentru $x = 2$ este 4.

- Pentru $N = 15$ şi $x = 4$

t	0	1	2	3	4	5	6	7	8	9	10	11	12	...
$f_{4,15}(t)$	1	4	1	4	1	4	1	4	1	4	1	4	1	...

avem perioada $r = 2$.

- Pentru $N = 15$ şi $x = 13$ avem

t	0	1	2	3	4	5	6	7	8	9	10	11	12	...
$f_{13,15}(t)$	1	13	4	7	1	13	4	7	1	13	4	7	1	...

deci perioada va fi $r = 4$.

Găsirea ordinului/perioadei

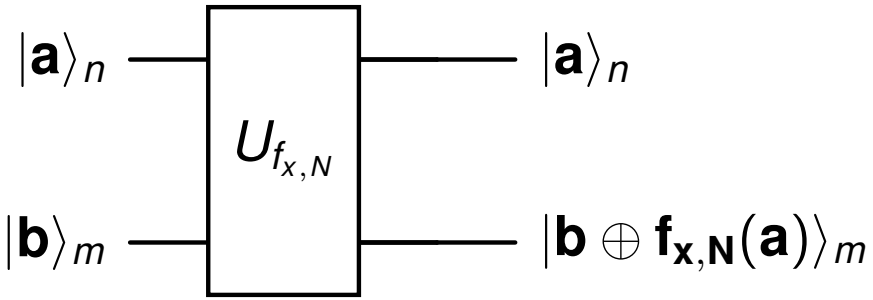
- Nu vom avea neaparat nevoie de toate valorile funcţiei, ci mai degraba ar trebui să găsim perioada funcţiei, adică cel mai mic număr r cu proprietatea că

$$f_{x,N}(r) = x^r \bmod N = 1$$

  iar dacă $f_{x,N}(r) = 1$, atunci $f_{x,N}(r+s) = f_{x,N}(s), \forall s$.
- Pentru numere mici precum $15, 21, 247$ se poate calcula relativ uşor perioada funcţiei, însă dacă N ar avea mai multe digits(spre exemplu, de ordinul sutelor) calculatorul clasic va avea nevoie de foarte mult timp şi foarte multe resurse pentru a găsi ordinul lui x şi a factoriza N.
- Cu ajutorul superpoziţiei, *calculatorul cuantic* ne va ajuta să evaluăm $f_{x,N}(t)$ pentru toate valorile t dintr-o singur apel de circuit.

Partea cuantică a algoritmului

Circuitul cuantic care implementează funcţia $f_{x,N}$ este următorul:



unde $U_{f_{x,N}}$ este operatorul unitar descris prin

$$U_{f_{x,N}}|\mathbf{a}\rangle_\mathbf{n} \otimes |\mathbf{b}\rangle_\mathbf{m} = |\mathbf{a}\rangle_\mathbf{n} \otimes |\mathbf{b} \oplus \mathbf{f_{x,N}(a)}\rangle_\mathbf{m} = |\mathbf{a}\rangle_\mathbf{n} \otimes |\mathbf{b} \oplus (\mathbf{x^a} \bmod \mathbf{N})\rangle_\mathbf{m}$$

unde $f_{x,N}(a) = x^a \bmod N$, iar $|\mathbf{a}\rangle_n$ şi $|\mathbf{b}\rangle_m$ sunt stări de $n-$qubiţi, respectiv $m-$qubiţi.

Cum valorile funcţiei $f_{x,N}$ vor fi mereu mai mici decât N, alegem $m = \log_2 N$, iar $n = \log_2^{N^2} = 2m$.

Dacă perioada r este o putere a lui 2 putem lua $n = m$.

Implementarea porţii $U_{f_{x,N}}$ cu ajutorul porţilor cuantice

Pentru a putea implementa $U_{f_{x,N}}$ cu ajutorul matricilor unitare, va trebui să "împărţim " $U_{f_{x,N}}$ în mai multe matrici unitare(porţi cuantice).

Spre exemplu, dacă scriu **a** în binar avem

$$\mathbf{a} = a_{n-1}a_{n-2}\ldots a_1 a_0$$

sau, formal, a scris ca un număr este

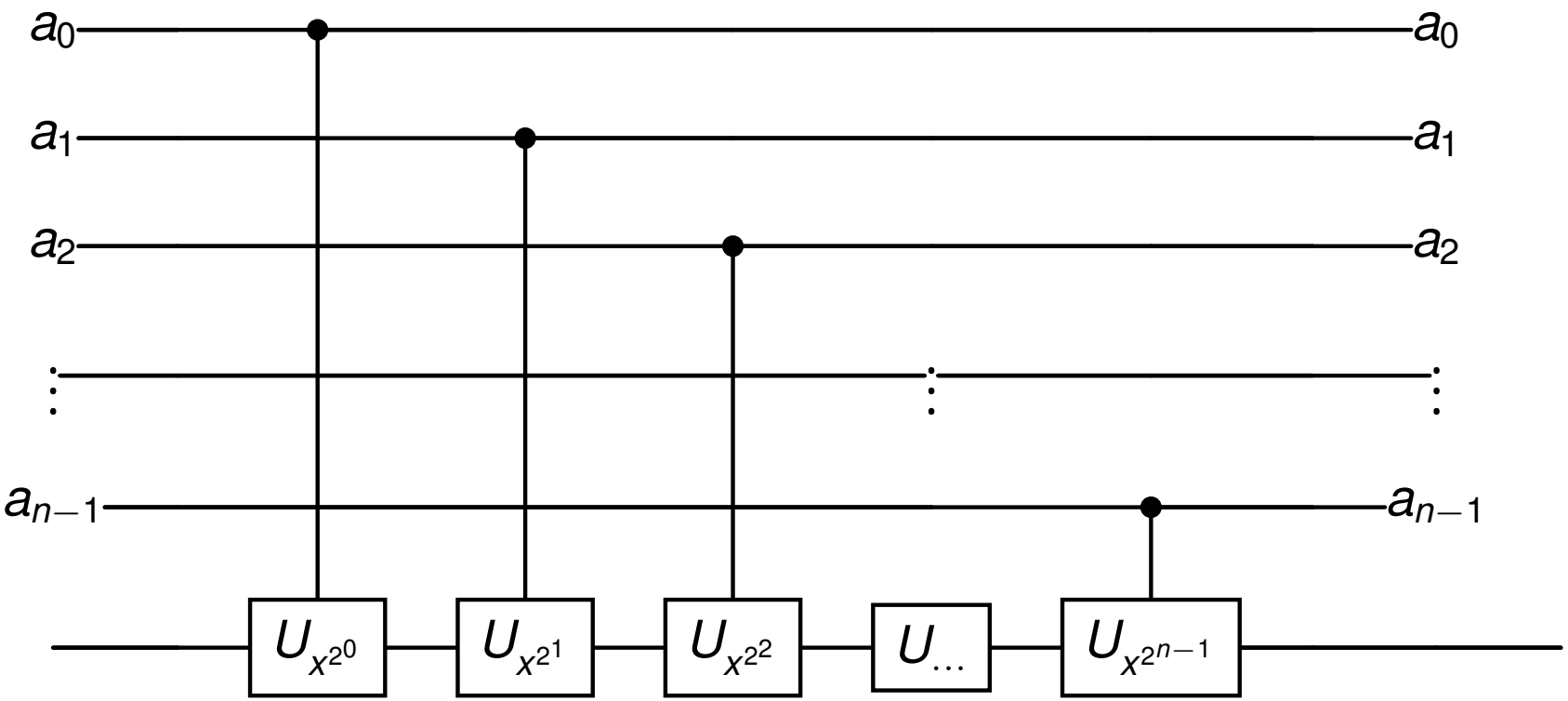
$$a = a_{n-1}2^{n-1} + a_{n-2}2^{n-2} + \ldots + a_1 2 + a_0.$$

Prin urmare, putem rescrie funcţia $f_{x,N}$ astfel

$$f_{x,N}(a) = x^a \bmod N = x^{a_{n-1}2^{n-1}} \cdot x^{a_{n-2}2^{n-2}} \cdot \ldots \cdot x^{a_1 2} \cdot x^{a_0} \bmod N.$$

Implementarea porţii $U_{f_{x,N}}$ cu ajutorul porţilor cuantice

Cu această idee, vom putea reprezenta $U_{f_{x,N}}$ cu o serie de porţi $Controlled - U_{x^{2^i}}$.



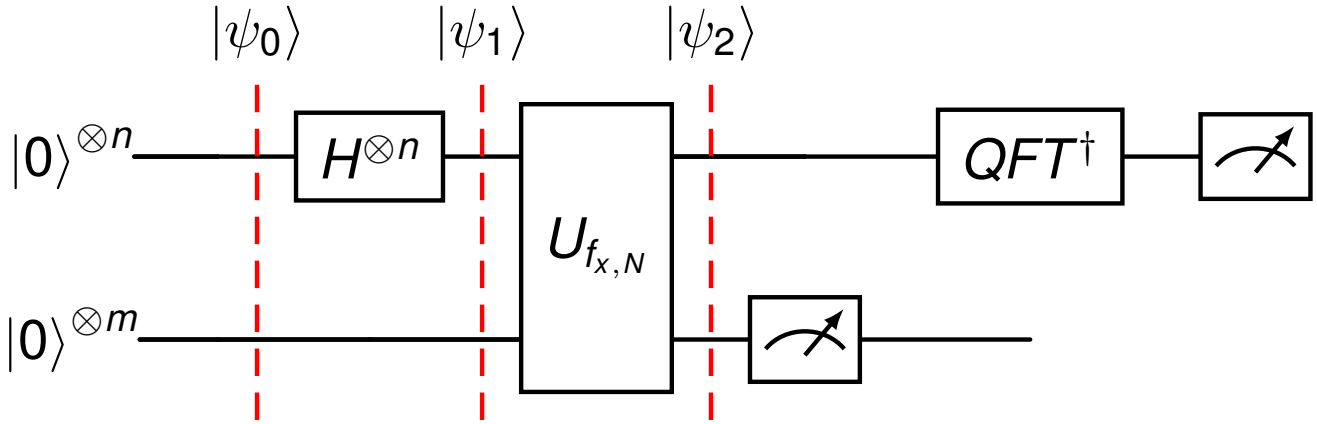
Algoritmul lui Shor

Paşii algoritmului lui Shor de determinare a perioadei sunt următorii:

1. Alegem starea iniţială: $|\psi_0\rangle = |0\rangle^{\otimes n} \otimes |0\rangle^{\otimes m}$
2. Crearea unei superpoziţii de stări aplicând Hadamard qubitilor din primul registru;
3. Implementarea funcţiei f ca o transformare cuantică (modular exponentiation);
4. Aplicarea transformatei Fourier cuantice inversă ($QFT^\dagger$) asupra primului registru;
5. Măsurare şi obţinerea perioadei r a funcţiei.

Algoritmul lui Shor

Algoritmul lui Shor cuantic poate fi implementat cu ajutorul următorului circuit:



unde $QFT^\dagger$ reprezintă inversa transformatei Fourier cuantice.

Starea inițială:

Starea calculatorului la începutul circuitului este

$$|\psi_0\rangle = |0\rangle^{\otimes n} \otimes |0\rangle^{\otimes m}.$$

Algoritmul lui Shor

Superpoziţia:

Aplicând operatorul Hadamard asupra primilor n qubiţi, vom găsi

$$|\psi_1\rangle = H^{\otimes n} \otimes |0\rangle^{\otimes m} = \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} |k\rangle_n \otimes |0\rangle_m,$$

unde, pentru simplitate, am notat cu $|\cdot\rangle_j$ o stare cu j qubiţi.

Aplicarea oracolului $U_{f_{x,N}}$:

$$|\psi_2\rangle = U_{f_{x,N}}|\psi_1\rangle = \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} U_{f_{x,N}}(|k\rangle_n \otimes |0\rangle_m) = \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} |k\rangle_n \otimes |x^k \bmod N\rangle_m$$

- Cum operatorul $U_{f_{x,N}}$ este liniar, el va acţiona asupra tuturor stărilor $|k\rangle_n \otimes |0\rangle_m$ simultan pentru toate cele 2^n valori ale lui k, generând şi toate puterile lui x simultan. Această caracteristică se numeşte **paralelism cuantic**.

Algoritmul lui Shor

Mai mult, dacă x^k mod N este periodică de perioadă r, atunci avem relaţia $x^k \equiv x^{k+r}$ mod N şi atunci funcţia $f_{x,N}(k) = x^k$ mod N va fi o funcţie periodică de perioadă r.

Pentru a vedea ce se obţine dacă am măsura stările de ieşire, vom rescrie $|\psi_2\rangle$ colectând termenii egali din registrul al doilea.

Cum x^k mod N are perioada r, vom nota $k = \alpha r + \beta$, unde $0 \leq \alpha \leq \frac{2^n}{r} - 1$, iar $0 \leq \beta \leq r - 1$. Înlocuind k în $|\psi_2\rangle$ şi folosind că x^β şi $x^{\alpha r+\beta}$ sunt echivalente modulo N, obţinem

$$|\psi_2\rangle = \frac{1}{\sqrt{2^n}} \sum_{\beta=0}^{r-1} \sum_{\alpha=0}^{\frac{2^n}{r}-1} |\alpha r + \beta\rangle_n \otimes |x^\beta \text{ mod } N\rangle_m,$$

Dacă măsurăm cel de-al doilea registru, vom obţine $x^0, x^1, \ldots, x^{r-1}$ cu probabilităţi egale.

Algoritmul lui Shor

Dacă rezultatul obţinut este $x^{\beta'}$, atunci starea după măsurare a calculatorului cuantic este

$$|\psi_3\rangle = \sqrt{\frac{r}{2^n}} \sum_{\alpha=0}^{\frac{2^n}{r}-1} |\alpha r + \beta'\rangle_n \otimes |x^{\beta'} \bmod N\rangle_m.$$

Găsirea perioadei utilizând $QFT^\dagger$:

Vom aplica transformata Fourier cuantică inversă asupra registrului de intrare (primii n qubiti).

Definiţia generală a acestei transformări este:

$$QFT^\dagger |x\rangle_n = \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} e^{-\frac{2i\pi}{N}xy} |y\rangle_n$$

Algoritmul lui Shor

Starea sistemului după aplicarea transformatei Fourier cuantice va fi următoarea

$$|\psi_4\rangle = QFT^\dagger|\psi_3\rangle = \sqrt{\frac{r}{2^n}}\sum_{\alpha=0}^{\frac{2^n}{r}-1}\left(\frac{1}{\sqrt{2^n}}\sum_{j=0}^{2^n-1}e^{-\frac{2\pi ij}{N}(\alpha r+\beta')}|j\rangle\right)|x^{\beta'} \bmod N\rangle.$$

Inversând ordinea de sumare avem

$$|\psi_4\rangle = \frac{1}{\sqrt{r}}\left[\sum_{j=0}^{2^n-1}\left(\frac{r}{2^n}\sum_{\alpha=0}^{\frac{2^n}{r}-1}e^{-\frac{2\pi i}{N}j\alpha r}\right)e^{-\frac{2\pi i}{N}j\beta'}|j\rangle_n\right]|x^{\beta'} \bmod N\rangle.$$

Cum $\frac{1}{N}\sum_{j=0}^{N-1}e^{\frac{2\pi i}{N}jk} = \begin{cases} 1, & \text{dacă } k \text{ este multiplu de } N \\ 0, & \text{în caz contrar} \end{cases}$ vom avea paranteza rotundă nenulă doar dacă $j = \frac{k2^n}{r}$, unde $k = 0, \ldots, r-1$.

Algoritmul lui Shor

Aşadar, dacă $j = k\frac{2^n}{r}$ vom avea

$$|\psi_4\rangle = \frac{1}{\sqrt{r}}\left(\sum_{k=0}^{r-1} e^{-\frac{2\pi i}{r}k\beta'}|\frac{k\cdot 2^n}{r}\rangle_n\right)|x^{\beta'} \bmod N\rangle_m$$

Măsurarea:

Dacă obţin valoarea $\frac{k'2^n}{r}$, unde $0 \leq k' \leq r-1$, atunci procedez astfel

- dacă $k' = 0$ ($j = 0$), nu ştiu nimic, şi atunci voi rula din nou partea cuantică.
- dacă $k' \neq 0$, impart $\frac{k'2^n}{r}$ la 2^n şi obţin valoarea $\frac{k'}{r}$ (fără să cunosc k', r).

Dacă $k' \neq 0$, utilizăm un algoritm cuantic găsesc forma raţională a lui $c = \frac{k'}{r}$, adică găsesc a, b astfel încât $c = a/b$ şi $gcd(a, b) = 1$.

În acest caz, ordinul lui x va fi b.

Exemplu - Factorizarea pentru $N = 15$

- https://qiskit.org/textbook/ch-algorithms/shor.html

Referinţe

1. M.A. Nielsen and I.L. Chuang- *Quantum computation and quantum information*, Cambridge University Press, 2010 (cap. 5,6) link.